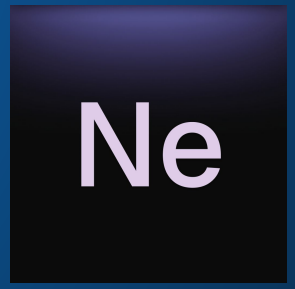
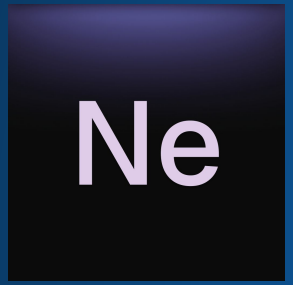
Low Level C++ Theory & Applications

Amlal El Mahrouss



I’m Amlal from Ne.app.



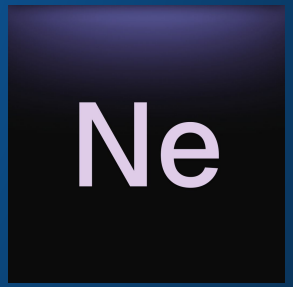
Email: amlal@ne-app.eu

Purpose of the talk:

→ In this talk we're going to layout techniques and decisions to take in order to successfully develop a low-level system in modern C++.

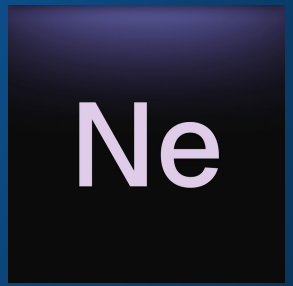
The standards involved:

→ C++17 and later.



On References:

- Unreference claims are not included in this talk for the sake of accuracy.
- P.S: These slides are available on GitHub. publications-and-more/talk.
- Thank you.



I: ‘Core’ Principles Covered.

Amlal El Mahrouss

Observation: What we will cover in this talk.

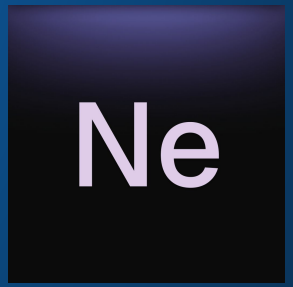
→ Common RAII Patterns.

→ When to use RAII?

→ List of Several RAII Containers.

→ Applications and Examples.

→ Error Handling and Graceful Exits.



II: Several Common RAII Patterns.

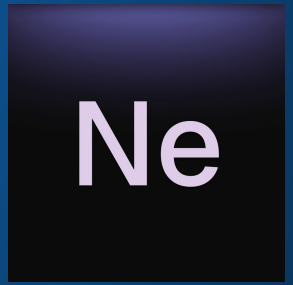
Amlal El Mahrouss

When to use RAII?

When using non-freestanding C++ RAII is mostly a ‘no-brainer’:

→ std::unique_ptr.
→ std::shared_ptr.

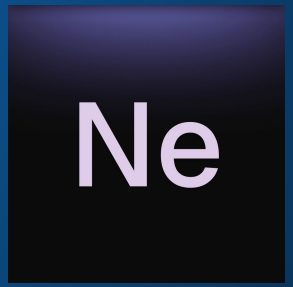
However, one may judge to roll its own container when deemed necessary, such as Open C++ Libraries, ‘tracked_ptr’, or Boost’s custom expansions of shared pointers operations such as delete_ptr.



When to use RAII?

An example:

Ant resource handling in HAL.dll and minoskrnl.exe:
It’s architecture makes use of the RAII pattern to manage resources that are deemed to be eligible for such operations.
Another example, file descriptors are when used by a routine are automatically freed after the routine finishes running.



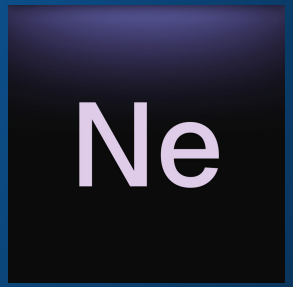
When to use RAII?

Example: A pool allocator in a graphics engine or high-throughput system.

This is a non-trivial operation, where the complexity of the algorithm may be exponential here. Thus per RAII, we will need to perform caching and implement a pool allocation algorithm.

This helps for:

- Linearity of the search.
- Searching time per algorithm complexity.



III: A List of Several RAII Containers and Patterns.

Amlal El Mahrouss

The OwnPtr, shared_ptr, and auto_ptr.

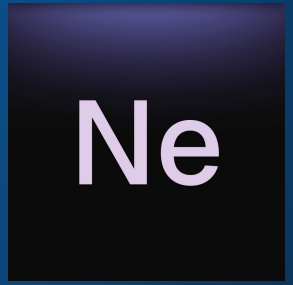
OwnPtr, A type introduced in WebKit, back when C++0x wasn't formalized yet.

Another example: Ne.app Ant and Ne.app NeSystem have their own RAII primitives.

And so does many C++ software…

Even some C based software does (thinking about the GCC extensions for that purpose)

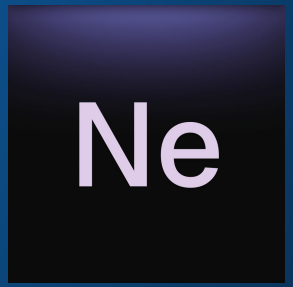
Source: Implementing RAII in pure C? - Stack Overflow



Design Matters!

After shipping and maintaining several software projects, I've come to this conclusion, and RAII containers are a prime example of this. What was once started with auto_ptr, had to be scaled to several refactors (unique, shared pointers) post C++11 due to the demand of more demanding containers.

I would love to talk more about how could one would design such software in the long-term, but I'm afraid It's not the scope of the talk.



IV: Applications and Examples Low Level C++.

Amlal El Mahrouss

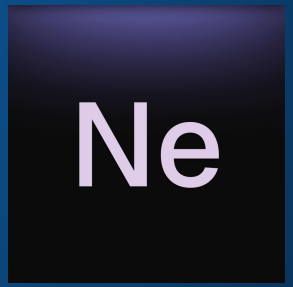
Example one: A Telnet Server

$ telnet telnet.example.com (Made up server host)

Now… One shall handle, the TUI, resources allocation, logging, etc…

What happens if the resource is busy? What happens if the NIC is being plugged off? Or the router crashes?

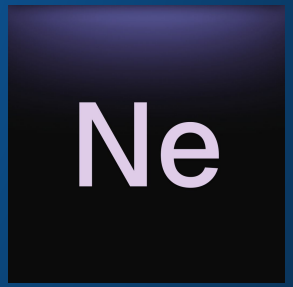
That’s why RAII is crucial, it helps handles those cases!



Example one: A Telnet Server

$ telnet telnet.example.com (Made up server host)

One may wrap the resources into a `Ref` container and pointers into a `RefPtr` container. Thus giving us the ability to handle the telnet program resources more effectively.

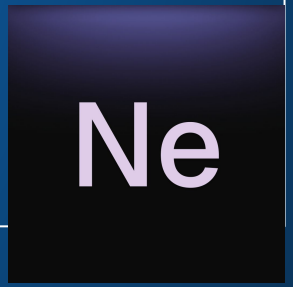


Example one: A Telnet Server

$ telnet telnet.example.com (Made up server host)

One may wrap the resources into a `Ref` container and pointers into a `RefPtr` container. Thus giving us the ability to handle the telnet program resources more effectively.

However, abuse of RAII concepts and patterns, will lead to issues with the program down the line. As it will become increasingly difficult



Example: System level allocators, and abstracting it into a RAII container.

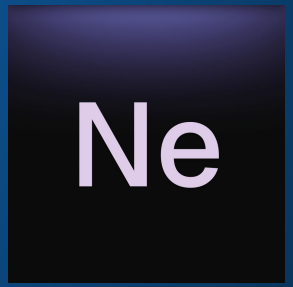
The following snippet is written in C:

VoidPtr x = MmAllocHeap(sz, flags, …);

This compiles in C, but could be improved for ‘higher level applications’*

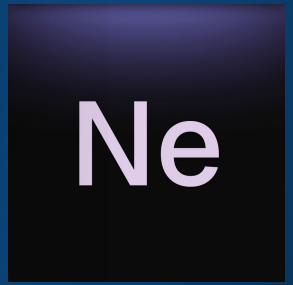
[1] An application that does not require manual management

Of the process’ memory.



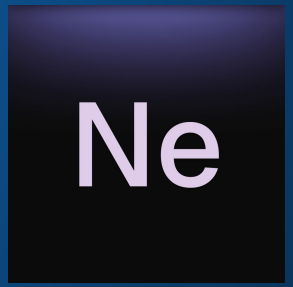
Example: System level allocators, and abstracting it into a RAII container.

```
VoidPtr x = MmAllocHeap(sz, flags, …);
if (x != nullptr) {
  /// Turn 'x' in a pool or whatever...
}
```



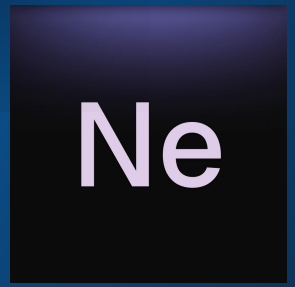
Example: System level allocators, and abstracting it into a RAII container.

Gets turned into this...



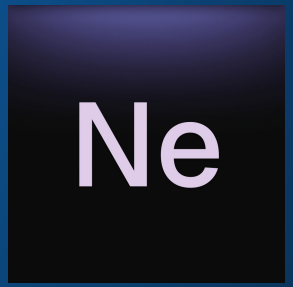
Example: System level allocators, and abstracting it into a RAII container.

```
/// Now the x is freed when doing function unwinding on the stack.
OwnPtr<VoidPtr> x = MmAllocHeap(sz, flags, …);
if (x) {
  /// Turn 'x' in a pool or whatever...
}
```



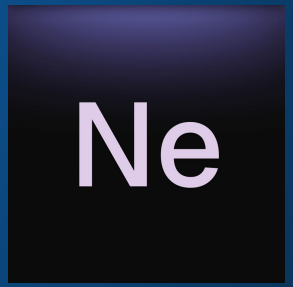
Example: System level allocators, and abstracting it into a RAII container.

This applies well for this example, under a specific caveat: The resource scope and lifetime has to be determined and predictable.



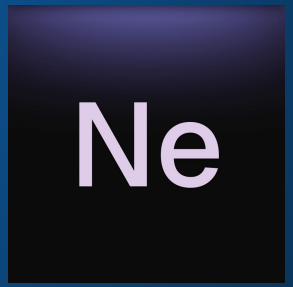
Example: System level allocators, and abstracting it into a RAII container.

Now several notes...



Example: System level allocators, and abstracting it into a RAII container.

Note: One may provide a deleter predicate on the container initialization for allocators that have non-trivial behavior and are not compatible with the standard OS allocator. E.g: COM, Qt...



V: Error Handling and Graceful Exits.

Amlal El Mahrouss

V/VI: Conclusion.

Amlal El Mahrouss